



Week 1: Environment Setup & Robotics Foundations

TURTLE Incubator — SO-ARM Software Stack

Program Overview

Our Mission

- Develop the next generation of leaders through hands-on robotics projects, competitions, and community service

Quick Facts

- Founded in 2013 the oldest and largest active robotics organization at TAMU
- 450+ active members across the org
- 18 student-led R&D projects, including SI's own SO-ARM software stack

SI in One Line

- A hands-on program that takes complete beginners from zero to programming and controlling a real robotic arm (SO-ARM) in 9 weeks

What SI Really Teaches You

- Not just following tutorials you'll learn to read hardware docs, debug real systems, and think like a robotics software engineer
- Every skill maps directly onto TURTLE's Advanced Projects, where this software runs on real competition robots

Presence & Conduct

- An officer or lab lead must be present whenever the lab is in use
- Disrespectful or unsafe behavior can result in disciplinary action or being asked to leave

Priority & Privileges

- Project tasks always take priority over personal projects for lab tools and space
- Misuse or unsafe practices will result in lab privileges being revoked

When In Doubt

- If you need training or supervision on a tool, ask an officer
- If common sense conflicts with anything here, ask an officer for help

Your Safety Comes First

- You're surrounded by people and resources who can help — if you're ever unsure, ask before you act
- Stop immediately and get clarification if you feel unsafe with a tool

Tools That Require Training

- Soldering iron, 3D printers, drill press, hand and power tools
- Any officer can provide tool-specific training

Required Safety Gear

- Closed-toe shoes, bottoms covering to the lower leg, a sleeved shirt, and safety glasses
- Loose hair and clothing must be secured up and out of the way



Meet the SI Team

Software Incubator Leads & Instructors

Julia Sopala - Development VP



Email: jusopala18@tamu.edu

Discord: juli.a9847



- BMEN 2027
- Meeting Days: TBD
- Experience:
 - OLSN (Prosthetic hand project)
 - R&D Engineering Internship
 - Biomedical Research Lab Experience
 - Hatchling Instructor
 - Former Hatchling
- Fun facts:
 - I want to be fluent in 4 languages
 - I'm 3/5th of the way there
 - I want to learn how to surf

Kaleb - SI Lead



Email: Hkaleb290@tamu.edu
Discord: thanos0818



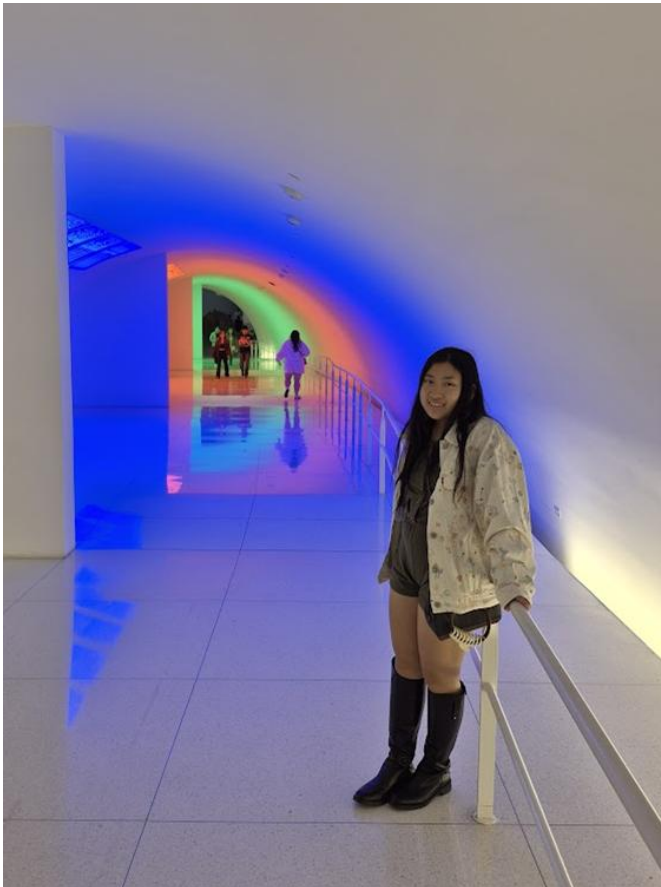
- ENGR (2028)
- Meeting Days: TBD
- Experience:
DIRT Software team lead (now project lead)
just a chill dude
- Fun facts:
rides a skateboard

Kayla Sam - SI Instructor



Email: kaysam@tamu.edu

Discord: kkaayyss



- ENGR 2029
- Meeting Days: TBD
- Experience:
 - FASH Electrical Team Lead
 - FTC Robotics
- Fun facts:
 - I have a twin brother
 - I sing in an A&M Choir
 - I'm living in a temporary apartment 🌸



Mohid Ismail - SI Instructor



Email: mohidismail@tamu.edu

Discord: bean.man.



- ELEN 2027
- Meeting Days:
 - the only one ig
- Experience:
 - VIRT
 - Optics Research Lab Experience
 - Former Hatchling
- Fun facts:
 - Have over 7000 hours on LoL
 - Lost a 3v1 to a goat once

Nathan Mersino - SI Instructor



Email: nathanmersino@tamu.edu
Discord: nathanmersino



- ELEN 2028
- Meeting Days:
 - TBD
- Experience:
 - POBS: Electrical Subteam Lead
 - Former Hatchling
 - Instrumentation Engineering/Transmitter Internship
- Fun facts:
 - In the TAMU Photography Club
 - 1220 Day Duolingo Streak





Icebreakers!

Icebreaker: Name and Fact



How It Works

- Share your name, major, year, and one fun fact about yourself

After Everyone Goes

- Pair up with someone who shared a similar fact and chat for a minute

TURTLE Introductions

- Meet the team leads, learn names, understand how the lab operates

Incubator Goals

- Build the complete software stack of a robotic arm — from zero to autonomous motion

Expectations

- Weekly hands-on deliverables you must demo before leaving each session

Semester Roadmap

- 9 weeks: setup → Python → kinematics → control → estimation → planning → ROS 2

SO-ARM Project Overview

- Every topic this semester maps directly to making the SO-ARM move smarter

Safety & Rules

- Always power down before touching wiring. Never force a joint. Ask before you unplug anything.



Robotics Software Architecture

What Makes a Robot Intelligent?

A robot without software is just a sculpture.

Three things every intelligent robot must do:

- Sense — read the world through cameras, encoders, and IMUs
- Decide — algorithms process that data and choose what action to take
- Act — motors, servos, and grippers execute the decision physically

This sense → decide → act loop runs hundreds of times per second.

The faster and more accurate it runs, the more intelligent the robot appears.

Software vs. Hardware Responsibilities



Hardware does:

- Move joints, resist gravity, handle electrical signals, enforce physical limits

Software does:

- Read sensors, compute what to do, send commands, catch errors, log data

The rule:

- Hardware sets the ceiling. Software determines how close you get to it.
- Bad software on good hardware = broken robot. Good software on bad hardware = still a broken robot.

Your job this semester is the software side — all of it.

Perception

- Reads raw encoder ticks, camera pixels, IMU data and turns them into useful state: "the arm is at 45°"

Planning

- Given a goal ("pick up the block"), figures out the sequence of positions to move through

Control

- Executes the plan by constantly comparing target vs actual position and correcting the error (PID)

Hardware Interface

- The lowest layer — translates software commands into actual voltage signals to the motor drivers

We will build each layer from scratch, one week at a time.



Development Environment

Installs — What You Need Today



Miniforge (conda) → runs LeRobot directly on your laptop, no Docker needed

- Windows: download the installer from github.com/conda-forge/miniforge, then use Miniforge Prompt
- Mac/Linux: run the Miniforge install script in Terminal (full steps in the SI Setup Guide)

Git

- Windows: git-scm.com/download/win | Mac: `brew install git` | Linux: `sudo apt install git`

VS Code

- Download at code.visualstudio.com, then install the Python extension

Verify everything works:

- `conda --version` `git --version` `code --version`

Linux is an open-source operating system used on 96% of the world's web servers and every major robotics platform.

Why robotics runs on Linux:

- ROS 2 (Robot Operating System) is Linux-native
- Real-time control, USB device access, and serial comms are much easier on Linux
- Most robotics libraries are written and tested for Linux first

Key commands you'll use constantly:

- `ls` `cd` `pwd` `mkdir` `rm` `cat` `grep` `chmod` `sudo` `apt install`

Conda creates separate Python environments, so each project gets exactly the packages it needs without breaking anything else on your laptop.

Environment

- An isolated folder with its own Python version and packages. Ours is called lerobot.

Why not Docker?

- Docker kept causing problems reaching the arms over USB, especially on Windows and Mac. Conda talks to the ports directly.

Commands you'll use:

- `conda activate lerobot` # enter the environment (every session)
- `conda deactivate` # leave it
- `conda env list` # list your environments

VS Code is our editor. Point it at your lerobot environment so your code runs with the right packages.

Install these extensions now:

- Python (Microsoft): IntelliSense, linting, debugger, and the environment picker
- GitLens: see who changed what and when, directly in the editor
- Robot Developer Extensions for ROS 2 (Ranch Hand Robotics): launch files and .msg/.srv types

Select your environment:

- Ctrl+Shift+P → 'Python: Select Interpreter' → pick the one named lerobot

Useful shortcuts:

- Ctrl+` open terminal | Ctrl+Shift+P command palette | Ctrl+Click jump to definition

Git saves a complete history of every change to your code. You can always go back.

The three commands you'll use every session:

- `git add .` # stage all changed files
- `git commit -m "msg"` # save a snapshot with a description
- `git push` # upload your commits to GitHub

Branches let you experiment safely:

- `git checkout -b my-feature` # new branch
- `git checkout main` # switch back to main

Pull Requests let teammates review your code before it merges into main.

SO-ARM Playground

What is the SO-ARM?

The SO-ARM100 is a 6-degree-of-freedom desktop robotic arm built from servo motors and 3D-printed parts.

6 degrees of freedom (DoF) means:

- 6 independent joints — each one rotates to position the arm in 3D space
- More DoF = more flexibility in how you reach a target position

What it can do:

- Pick and place objects, follow trajectories, react to sensor feedback in real time

What powers it:

- Feetech STS3215 servo motors — each has a built-in encoder and PID controller on-chip

You installed LeRobot with pip, so you don't need to clone anything to use it. To read the source code:

- `git clone https://github.com/huggingface/lerobot`
- `src/lerobot/robots/so_follower/` # the follower arm (S0100 and S0101)
- `src/lerobot/teleoperators/so_leader/` # the leader arm
- `src/lerobot/motors/feetech/` # low-level servo driver
- `src/lerobot/scripts/` # all the lerobot-* commands

Everything you touch this semester lives in `so_follower`, `so_leader`, and `motors/feetech`.

Key Files You Need to Know



robots/so_follower/so_follower.py

- The follower robot class. Connects, reads joint positions, and sends commands to the SO-ARM.

robots/so_follower/config_so_follower.py

- The settings you pass in: port, id, max_relative_target (a safety limit), and PID gains.

teleoperators/so_leader/so_leader.py

- The leader arm class. Reads the joints you move by hand during teleoperation.

motors/feetech/feetech.py

- Low-level serial driver for the Feetech servos. Handles read/write over USB-to-serial.

How the SO-ARM Code is Organized



The codebase has three levels. Understand which level you're working at:

Level 1: Motor Driver (`feetech.py`)

- Speaks directly to servo hardware over serial. Reads raw encoder ticks, writes target positions.

Level 2: Robot Class (`so_follower.py`)

- Wraps the motor driver. Uses joint names instead of motor IDs, applies calibration, reads full arm state.

Level 3: Commands (`lerobot-calibrate`, `lerobot-teleoperate`, ...)

- What you type in the terminal. Calls the robot class to calibrate, teleoperate, record, or replay.

When something breaks, start at Level 3 and trace down. Don't start at the driver.

Understanding so_follower.py: The Robot Class



This file is the heart of everything. Four methods you must understand:

connect()

- Opens the USB serial port, checks that every motor responds, and loads your calibration file.

get_observation()

- Reads every joint at once. Returns a dict like {'shoulder_pan.pos': 12.3, 'elbow_flex.pos': 90.2, ...}

send_action(action)

- Sends target positions to the joints. Action is a dict of 'joint_name.pos' → angle in degrees.

disconnect()

- Releases the port and turns torque off so the arm goes limp. Support the arm, and call it when you're done.

There's no YAML file to edit. You pass settings in when you create the robot:

- `config = SO101FollowerConfig(`
- `port='YOUR_FOLLOWER_PORT', # from lerobot-find-port`
- `id='follower_arm', # must match your calibration name`
- `max_relative_target=10, # optional: max degrees per step`
- `)`

Motor IDs are fixed in `so_follower.py`:

- shoulder_pan 1, shoulder_lift 2, elbow_flex 3, wrist_flex 4, wrist_roll 5, gripper 6
- Each ID must match the ID stored on the servo. `lerobot-setup-motors` sets these for you.

Running Existing Examples: Step by Step



Step 1 Connect the arms

- Plug in both USB cables and the follower's power supply, then find the ports: `lerobot-find-port`

Step 2 Activate your environment

- `conda activate lerobot`

Step 3 Run teleoperation (Windows: one line, no \)

- `lerobot-teleoperate \`
- `--robot.type=so101_follower --robot.port=YOUR_FOLLOWER_PORT \`
- `--robot.id=follower_arm --teleop.type=so101_leader \`
- `--teleop.port=YOUR_LEADER_PORT --teleop.id=leader_arm`

Step 4 What you should see

- Moving the leader arm moves the follower arm in real time.

Step 5 Stop cleanly

- Ctrl+C in terminal. Torque turns off when it exits, so support the follower arm before you stop.

The robot streams state data continuously while connected. Here's how to read it in Python:

- `from lerobot.robots.so_follower import SO101Follower, SO101FollowerConfig`
- `config = SO101FollowerConfig(port='YOUR_FOLLOWER_PORT', id='follower_arm')`
- `robot = SO101Follower(config)`
- `robot.connect()`

- `# Read all joint positions (dict: degrees, gripper is 0 to 100)`
- `obs = robot.get_observation()`
- `print(obs) # e.g. {'shoulder_pan.pos': 12.3, 'shoulder_lift.pos': -45.0, ...}`

- `# Read current (raw units: higher means more load on that motor)`
- `currents = robot.bus.sync_read('Present_Current')`

Sending Commands to the Arm

Sending a position command tells each joint where to go. The on-chip PID handles getting there.

- `# Move to a specific pose (angles in degrees, gripper 0 to 100)`
- `action = {`
- `'shoulder_pan.pos': 0.0, # centered`
- `'shoulder_lift.pos': -45.0, # tilted back`
- `'elbow_flex.pos': 90.0, # elbow bent 90°`
- `'wrist_flex.pos': 0.0,`
- `'wrist_roll.pos': 0.0,`
- `'gripper.pos': 20.0,`
- `}`
- `robot.send_action(action)`
- `robot.disconnect() # when you're done`

Three safe places to modify behavior, from safest to most impactful:

1 Command-line arguments (safest)

- Change ports, ids, or the control rate. No code changes needed.
- `lerobot-teleoperate ... --fps=10` # default is 60

2 Robot config (safe)

- `max_relative_target` caps how far a joint can move per step, a built-in speed limit
- `lerobot-teleoperate ... --robot.max_relative_target=5`

3 Your own Python script (careful)

- Wrap `send_action()` with your own speed limiting, smoothing, or safety checks
- This is where real engineering happens, but test with small moves first



Demonstration

Demonstration — Move SO-ARM Using Provided Software



Watch carefully — you will repeat every step of this in the hands-on.

What we'll show:

- 1. Connect arms, find ports with lerobot-find-port, activate the lerobot environment
- 2. Run lerobot-teleoperate and move the leader arm by hand
- 3. Print live joint positions to terminal while arm moves
- 4. Send a hardcoded position command and watch the arm move to it
- 5. Read motor current to see which joints are under load

Demonstration — Exploring Robot States and Commands



After moving the arm, we'll dig into what the robot is actually reporting:

Present_Position

- Each joint's current angle in degrees. This is ground truth for where the arm is.

Present_Velocity

- How fast each joint is moving. Near zero = arm is still. High value = arm is swinging fast.

Present_Current

- Electrical current drawn. High current on a stationary joint = it's fighting gravity or hitting a limit.

Hardware Error Status

- Non-zero means a motor fault. Overload, overheat, or encoder error. Always check this first when something is wrong.



Hands-On

Goal: get LeRobot installed on your laptop and find your arms' ports.

1. Install Miniforge and open a terminal

- Follow Step 1 of the SI Setup Guide, then open Miniforge Prompt (Windows) or Terminal (Mac/Linux)

2. Create and activate the environment

- `conda create -y -n lerobot python=3.12`
- `conda activate lerobot`
- `conda install -y ffmpeg -c conda-forge`

3. Install LeRobot (Windows: use double quotes instead of single)

- `pip install 'lerobot[core_scripts,feetech]'`

4. Find your ports (arms plugged in, follower powered)

- `lerobot-find-port`
- Unplug one arm when asked. Windows: COM3 | Mac: `/dev/tty.usbmodem...` | Linux: `/dev/ttyACM0`

Hands-On: Calibrate and Set Up VS Code



Goal: calibrate both arms and open VS Code in your lerobot environment.

1. Verify the install

- `python -c "import lerobot; print(lerobot.__version__)"`

2. Calibrate the follower, then the leader (Windows: one line each, no \)

- `lerobot-calibrate --robot.type=so101_follower \`
- `--robot.port=YOUR_FOLLOWER_PORT --robot.id=follower_arm`
- `lerobot-calibrate --teleop.type=so101_leader \`
- `--teleop.port=YOUR_LEADER_PORT --teleop.id=leader_arm`
- Center every joint and press Enter, then move each joint through its full range and press Enter

3. Open in VS Code (optional but recommended)

- `Ctrl+Shift+P` → 'Python: Select Interpreter' → pick lerobot

Goal: make the arm move and read live joint data from Python.

1. Run teleoperation

- `lerobot-teleoperate \`
- `--robot.type=so101_follower --robot.port=YOUR_FOLLOWER_PORT \`
- `--robot.id=follower_arm --teleop.type=so101_leader \`
- `--teleop.port=YOUR_LEADER_PORT --teleop.id=leader_arm`
- Move the leader arm. The follower should mirror it in real time. Ctrl+C to stop.

2. Read joint positions in a Python script (stop teleop first, it holds the port)

- `from lerobot.robots.so_follower import SO101Follower, SO101FollowerConfig`
- `config = SO101FollowerConfig(port='YOUR_FOLLOWER_PORT', id='follower_arm')`
- `robot = SO101Follower(config)`
- `robot.connect()`
- `print(robot.get_observation())`

3. Run `robot.bus.disable_torque()`, then move joints by hand and print again to watch the numbers change



Deliverable

Deliverable: Successfully Run and Modify an Existing SO-ARM Example



You must demonstrate all four of the following to a lab lead before leaving:

1 LeRobot installed and ports found

- Show your (lerobot) prompt and both port names from lerobot-find-port

2 Both arms calibrated

- lerobot-calibrate finished for follower_arm and leader_arm

3 Teleoperation working

- lerobot-teleoperate runs and the follower mirrors the leader

4 Parameter modified and explained

- Change one argument (like --fps or --robot.max_relative_target), re-run, and explain what changed and why